

Tutorial-5

Linked List

1. [Write a function to delete a Linked List](#)

Iterate through the linked list and delete all the nodes one by one. The main point here is not to access the next of the current pointer if the current pointer is deleted.

```
/* Function to delete the entire linked list */
void deleteList(Node** head_ref)
{
    /* deref head_ref to get the real head */
    Node* current = *head_ref;
    Node* next = NULL;

    while (current != NULL)
    {
        next = current->next;
        free(current);
        current = next;
    }

    /* deref head_ref to affect the real head back
    in the caller. */
    *head_ref = NULL;
}
```

2. [Remove duplicates from a sorted linked list](#)

For example if the linked list is 11->11->11->21->43->43->60 then removeDuplicates() should convert the list to 11->21->43->60.

Algorithm: Traverse the list from the head (or start) node. While traversing, compare each node with its next node. If the data of the next node is the same as the current node then delete the next node. Before we delete a node, we need to store the next pointer of the node

```
/* The function removes duplicates from a sorted list */
void removeDuplicates(Node* head)
{
    /* Pointer to traverse the linked list */
    Node* current = head;

    /* Pointer to store the next pointer of a node to be deleted*/
```

```

Node* next_next;

/* do nothing if the list is empty */
if (current == NULL)
    return;

/* Traverse the list till last node */
while (current->next != NULL)
{
    /* Compare current node with next node */
    if (current->data == current->next->data)
    {
        /* The sequence of steps is important*/
        next_next = current->next->next;
        free(current->next);
        current->next = next_next;
    }
    else /* Only advance if no deletion */
    {
        current = current->next;
    }
}
}

```

METHOD 2

Alternate Approach: Create a pointer ptr that will point towards the first occurrence of every element and another pointer temp which will iterate to every element and when the value of the previous pointer is not equal to the temp pointer, we will set ptr->next = temp. But remember, before setting ptr->next = temp, you should free all the subsequent nodes having the same value. *Refer to the first question for the deletion (freeing) step.*

With this approach, if any value is repeated multiple times (more than twice), all the subsequent nodes having the same value can be excluded together. i.e. unlike the first approach, we need to reassign the pointers here exactly once (even if a value is repeated more than twice).

3. [Remove duplicates from an unsorted linked list](#)

For example if the linked list is 12->11->12->21->41->43->21 then removeDuplicates() should convert the list to 12->11->21->41->43.

METHOD 1 (Using two loops)

This is the simple way where two loops are used. Outer loop is used to pick the elements one by one and the inner loop compares the picked element with the rest of the elements.

```

/* Function to remove duplicates from a unsorted linked list */
void removeDuplicates(struct Node* start)
{
    struct Node *ptr1, *ptr2, *dup;
    ptr1 = start;

    /* Pick elements one by one */
    while (ptr1 != NULL && ptr1->next != NULL) {
        ptr2 = ptr1;
        /* Compare the picked element with rest of the elements */
        while (ptr2->next != NULL) {
            /* If duplicate then delete it */
            if (ptr1->data == ptr2->next->data) {
                /* sequence of steps is important here */
                dup = ptr2->next;
                ptr2->next = ptr2->next->next;
                delete (dup);
            }
            else /* This is tricky */
                ptr2 = ptr2->next;
        }
        ptr1 = ptr1->next;
    }
}

```

Time Complexity: $O(n^2)$

METHOD 2 (Using Sorting: Merge Sort on Linked List)

An efficient solution. *If merge sort has been covered in the lectures, we can discuss this solution else, we can skip.*

In general, Merge Sort is the best-suited sorting algorithm for sorting linked lists efficiently.

1) Sort the elements using Merge Sort. $O(n \log n)$

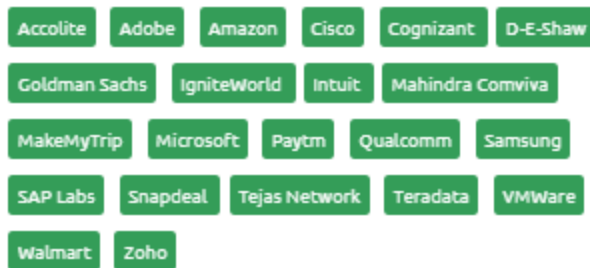
2) Remove duplicates in linear time using the [algorithm for removing duplicates in sorted Linked List. \$O\(n\)\$](#)

Please note that this method doesn't preserve the original order of elements.

Overall Time Complexity: $O(n \log n)$

4. [Reverse a linked list](#)

Given a pointer to the head node of a linked list, the task is to reverse the linked list (*without using any auxiliary space*).



If auxiliary space is allowed, one can use a Stack to generate the reversed list. We can discuss this point, if the stack data-structure has been covered in the class.

Examples:

Input1: 1->2->3->4->NULL

Output1: 4->3->2->1->NULL

Input2: NULL

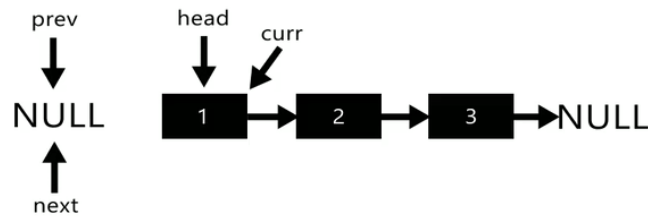
Output2: NULL

Input3: 1->NULL

Output3: 1->NULL

Iterative Method

1. Initialize three pointers prev as NULL, curr as head and next as NULL.
2. Iterate through the linked list. In loop, do the following.
 - // Before changing next of current,
 - // store next node
 - next = curr->next
 - // Now change next of current
 - // This is where actual reversing happens
 - curr->next = prev
 - // Move prev and curr one step forward
 - prev = curr
 - curr = next

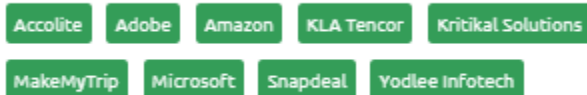


```

while (current != NULL)
{
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}
*head_ref = prev;

```

5. Given a singly linked list of characters, write a function that returns true [if the given list is a palindrome](#), else false. *<Auxiliary space is not allowed>*



If auxiliary space is allowed, we can use a Stack to store the elements of the linked list. Then, we can compare the elements in the Stack with the elements of the original linked list. If all the elements match, it is a palindrome, else not. We can discuss this point, if the stack data-structure has been covered in the class.

Iterative solution: By reversing the list

This method takes $O(n)$ time and $O(1)$ extra space.

1) [Get to the middle of the linked list.](#)

2) Reverse the second half of the linked list. ← *Refer to the previous question*

3) Check if the first half and second half are identical.

4) If there is a need to print the original list, construct the original linked list by reversing the second half again and attaching it back to the first half.

Que.6 Intersection Point in Y Shaped Linked Lists

Example 1:

Input:

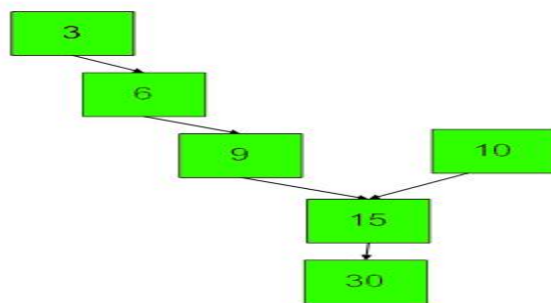
LinkedList1 = 3->6->9->common

LinkedList2 = 10->common

common = 15->30->NULL

Output: 15

Explanation:



Solution: <https://www.youtube.com/watch?v=7byKXAhxyM>
<https://www.youtube.com/watch?v=lunWIOjj0Cg>

7. Delete last occurrence of an item from a linked list <Auxiliary space is not allowed>

Example:

Input: 1->2->3->5->2->10, key = 2

Output: 1->2->3->5->10

The idea is to traverse the linked list from beginning to end. While traversing, keep track of the last occurrence key. Once you know the address of the node having the last occurrence of the key, [you can delete the node at the given address](#) (refer to the last tutorial, Que5: Delete without head pointer). The node having the last occurrence can be deleted by copying the data of the next node and then deleting the next node.

<Take note that this copy method will not work if the key occurs at the last element. You have to handle this scenario separately. The code below handles this special case separately.>

```
// Function to delete the last occurrence
void deleteLast(struct Node* head, int x)
{
    struct Node *temp = head, *ptr = NULL;
    while (temp) {

        // If found key, update
        if (temp->data == x)
            ptr = temp;
        temp = temp->next;
    }

    // If the last occurrence is the last node
    if (ptr != NULL && ptr->next == NULL) {
        temp = head;
        while (temp->next != ptr)
            temp = temp->next;
        temp->next = NULL;
    }

    // If it is not the last node
    if (ptr != NULL && ptr->next != NULL) {
        ptr->data = ptr->next->data;
        temp = ptr->next;
        ptr->next = ptr->next->next;
        free(temp);
    }
}
```

An alternate approach that can handle all cases together: Instead of storing the address of the node having the last occurrence, you should store the address of the preceding node (ptr) to it. Thereafter, you can delete the subsequent node to ptr without the need of copying.

8. [XOR Linked List – A Memory Efficient Doubly Linked List](#)

Accolite Amazon GE

An ordinary Doubly Linked List requires space for two address fields to store the addresses of previous and next nodes.

A memory-efficient version of Doubly Linked List can be created using only one space for the address field with every node. This memory efficient Doubly Linked List is called XOR Linked List or Memory Efficient as the list uses bitwise XOR operation to save space for one address. In the XOR linked list, instead of storing actual memory addresses, every node stores the XOR of addresses of previous and next nodes.

[Click this link to see an example and the implementation of XOR Linked List.](#)

[Insertion and deletion in an XOR Linked List](#)

9. How do you find the third node (or the k^{th} node) from the end in a singly linked list?

Accolite Adobe Amazon Citicorp Epic Systems FactSet
Hike MAQ Software Monotype Solutions Morgan Stanley
OVO Rooms Qualcomm Samsung Snapdeal

The main challenge here is to solve the problem in just one pass. That means, you can not traverse the linked list again and you cannot traverse backward because it's a singly linked list.

Here, we'll use the **tortoise and hare algorithm** to solve this problem. This is the same algorithm that we have used to find the middle element of the linked list in one pass.

The algorithm uses two pointers, fast and slow. The slow pointer starts when the fast pointer is reached to the k^{th} node. For example, in order to find the 3rd element from the last, the second pointer should start when the first pointer has reached the third element.

After that, both pointers will keep moving one step at a time until the first pointer points to `null`, which signals the end of the linked list. At this time, the second pointer is pointing to the 3rd or k^{th} node from the last.

10. [Partitioning a linked list around a given value and keeping the original order](#)

Given a linked list and a value x , partition it such that all nodes less than x come first, then all nodes with a value equal to x , and finally nodes with a value greater than or equal to x . Also, the original relative order of the nodes in each of the three partitions should be preserved.

Examples:

Input : 1->4->3->2->5->2->3, $x = 3$

Output: 1->2->2->3->3->4->5

Input : 1->4->2->10, $x = 3$

Output: 1->2->4->10

Input : 10->4->20->10->3, $x = 3$

Output: 3->10->4->20->10

Algorithm:

- Initialize first and last nodes of below three linked lists as NULL.
 1. Linked list of values smaller than x .
 2. Linked list of values equal to x .
 3. Linked list of values greater than x .
- Now iterate through the original linked list. If a node's value is less than x then append it at the end of the smaller list. If the value is equal to x , then at the end of the equal list. And if a value is greater, then at the end of the greater list.
- Now concatenate three lists.

Solution: https://www.youtube.com/watch?v=iLhRF71D3_w

11. [Rearrange linked list so that it has alternating high and low values](#)

Amazon

OYO Rooms

Given a linked list of integers, rearrange it such that every second node of the linked list is greater than its left and right nodes. In other words, rearrange the linked list node in alternating high-low.

Assume no duplicate nodes are present in the linked list. Several lists might satisfy the constraints; we need to print any one of them

Examples:**Input:** 1 → 2 → 3 → 4 → 5 → 6 → 7**Output:** 1 → 3 → 2 → 5 → 4 → 7 → 6**Input:** 9 → 6 → 8 → 3 → 7**Output:** 6 → 9 → 3 → 8 → 7**Input:** 6 → 9 → 2 → 5 → 1 → 4**Output:** 6 → 9 → 2 → 5 → 1 → 4**Algorithm:**

The idea is to start from the second node in the linked list and advance two nodes in each iteration of the loop. If the previous node is greater than the current node, swap their values. Similarly, if the next node is greater than the current node, exchange both values. At the end of the loop, we will get the desired linked list that satisfies the given constraints.

```
void swap(struct Node *first, struct Node *second)
{
    int temp = first->data;
    first->data = second->data;
    second->data = temp;
}

// Rearrange the linked list so that it has alternating high, low values
void rearrange(struct Node *head)
{
    // empty list
    if (head == NULL) {
        return;
    }

    struct Node* prev = head;
    struct Node* curr = head->next;

    // start from the second node
    while (curr)
    {
        // if the previous node is greater than the current node, swap their values
        if (prev->data > curr->data) {
            swap(prev, curr);
        }

        // if the next node is greater than the current node, swap their values
        if (curr->next && curr->next->data > curr->data) {
            swap(curr->next, curr);
        }
    }
}
```

```

    }

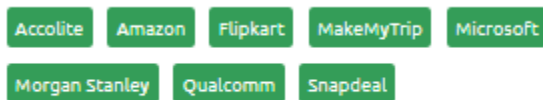
    // update `prev` and `curr` node
    prev = curr->next;

    if (!curr->next) {
        break;
    }

    curr = curr->next->next;
}
}

```

Q.12 Add two numbers represented by linked lists



The sum list is a linked list representation of the addition of two input numbers **from the last**.

Example 1:

Input:

N = 2

valueN[] = {4,5}

M = 3

valueM[] = {3,4,5}

Output: 3 9 0

Explanation: For the given two linked list (4 5) and (3 4 5), after adding the two linked list resultant linked list will be (3 9 0).

Example 2:

Input:

N = 2

valueN[] = {6,3}

M = 1

valueM[] = {7}

Output: 7 0

Explanation: For the given two linked list (6 3) and (7), after adding the two linked list resultant linked list will be (7 0).

Solution: https://www.youtube.com/watch?v=HzWKOp0C_aU